

NovaStore E-Ticaret Veri Yönetim Sistemi Projesi

Veri Tabanı ve SQL – Berke Mert ÖZTÜRK

BÖLÜM 1: Veri Tabanı Tasarımı (DDL)

Bu bölümde öncelikle tüm verilerimi içinde barındıracak olan ana veri tabanımı (NovaStoreDB) oluşturdum. Ardından yönergedeki "önce ana tablolar, sonra bağımlı tablolar" kuralına uyarak Categories ve Customers tablolarını kurdum. Daha sonra bu tablolara Foreign Key ile bağlanan Products, Orders ve en detaylı ara tablo olan OrderDetails tablolarını oluşturdum.

```
-- 1. Tüm projemi içine kuracağım ana veri tabanını oluşturun.
```

```
CREATE DATABASE NovaStoreDB;
```

```
GO
```

```
-- Tablolar yanlış yere açılmasın diye oluşturduğum veri tabanının  
içine giriyorum.
```

```
USE NovaStoreDB;
```

```
GO
```

```
-- 2. Ana Tablolar
```

```
-- Ürünleri gruplamak için önce kategoriler tablosunu açtım. ID  
otomatik artsın, isim boş kalmasın.
```

```
CREATE TABLE Categories (  
    CategoryID int IDENTITY(1,1) PRIMARY KEY,  
    CategoryName varchar(50) NOT NULL  
);
```

```
-- Müşteriler tablosu. Kimsenin e-postası diğeriyle aynı olmasın  
diye Email'e UNIQUE verdim.
```

```
CREATE TABLE Customers (  
    CustomerID int IDENTITY(1,1) PRIMARY KEY,  
    FullName varchar(50),  
    City varchar(20),
```

```

    Email varchar(100) UNIQUE
);

-- 3. Bağımlı Tablolar
-- Ürünler tablosu. Hangi kategoriye ait olduğunu bilmek için
CategoryID'yi Foreign Key olarak bağladım.
-- Stok girilmeyi unutulursa sistem otomatik 0 saysın diye DEFAULT 0
kuralını ekledim.
CREATE TABLE Products (
    ProductID int IDENTITY(1,1) PRIMARY KEY,
    ProductName varchar(100) NOT NULL,
    Price decimal(10,2),
    Stock int DEFAULT 0,
    CategoryID int FOREIGN KEY REFERENCES Categories(CategoryID)
);

-- Siparişler tablosu. Bu siparişi kim verdi bulabilmek için
CustomerID'yi müşterilere bağladım.
-- Tarih girilmezse o anki zamanı otomatik alsın diye DEFAULT
GETDATE() kullandım.
CREATE TABLE Orders (
    OrderID int IDENTITY(1,1) PRIMARY KEY,
    CustomerID int FOREIGN KEY REFERENCES Customers(CustomerID),
    OrderDate datetime DEFAULT GETDATE(),
    TotalAmount decimal(10,2)
);

-- 4. Ara Tablo
-- En detaylı yapı burası. Hangi siparişte hangi üründen alınmış
görebilmek için iki tane Foreign Key ile diğer tablolara bağladım.

```

```

CREATE TABLE OrderDetails (

    DetailID int IDENTITY(1,1) PRIMARY KEY,

    OrderID int FOREIGN KEY REFERENCES Orders(OrderID),

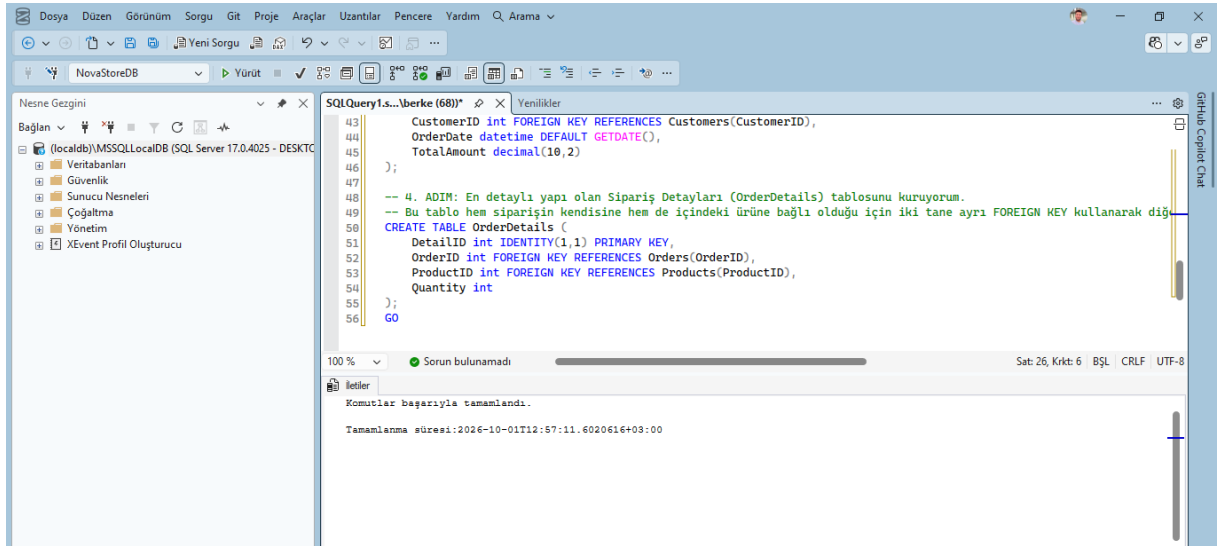
    ProductID int FOREIGN KEY REFERENCES Products(ProductID),

    Quantity int

);

GO

```



BÖLÜM 2: Veri Girişi (DML - Insert)

Sistemimi test edebilmek için tablolarıma örnek veriler (Dummy Data) ekledim. 5 farklı kategori, 12 adet ürün, 6 farklı müşteri, 10 adet sipariş ve bu siparişlerin detaylarını sisteme girdim. Testlerde kullanabilmek için 'Ahmet Yılmaz' müşterisini ve eski tarihli siparişleri özellikle ekledim.

-- Test için 5 tane örnek kategori giriyorum.

```

INSERT INTO Categories (CategoryName) VALUES ('Elektronik'),
('Giyim'), ('Kitap'), ('Kozmetik'), ('Ev ve Yaşam');

```

-- Kategorilere göre 12 tane ürün belirledim, fiyat ve stoklarını ayarladım.

```

INSERT INTO Products (ProductName, Price, Stock, CategoryID) VALUES
('Akıllı Telefon', 25000.00, 15, 1), ('Laptop', 45000.00, 8, 1),
('Bluetooth Kulaklık', 1500.00, 30, 1),

```

```

('Kışlık Mont', 2500.00, 50, 2), ('Spor Ayakkabı', 1800.00, 12, 2),
('Bilim Kurgu Romanı', 150.00, 100, 3),

('SQL Eğitim Kitabı', 250.00, 18, 3), ('Güneş Kremi', 400.00, 60,
4), ('Parfüm', 1200.00, 10, 4),

('Çalışma Masası', 3000.00, 5, 5), ('Masa Lambası', 500.00, 25, 5),
('Ergonomik Koltuk', 4500.00, 7, 5);

-- 6 tane sahte müşteri kaydı açıyorum. İleride sorgularda lazım
olacağı için Ahmet Yılmaz'ı özellikle ekledim.

INSERT INTO Customers (FullName, City, Email) VALUES

('Ahmet Yılmaz', 'İstanbul', 'ahmetyilmaz@email.com'), ('Ayşe Kaya',
'Ankara', 'aysekaya@email.com'),

('Mehmet Demir', 'İzmir', 'mehmetdemir@email.com'), ('Fatma Çelik',
'Bursa', 'fatmacelik@email.com'),

('Ali Can', 'Antalya', 'alican@email.com'), ('Zeynep Şahin',
'Eskişehir', 'zeynepsahin@email.com');

-- Müşterilerin verdiği siparişleri farklı tarihlerde olacak şekilde
sisteme giriyorum.

INSERT INTO Orders (CustomerID, OrderDate, TotalAmount) VALUES

(1, '2023-10-01 10:30:00', 26500.00), (1, '2023-10-05 14:00:00',
150.00), (2, '2023-10-02 09:15:00', 45000.00),

(3, '2023-10-03 16:45:00', 2500.00), (4, '2023-10-10 11:00:00',
3000.00), (5, '2023-10-12 13:20:00', 1600.00),

(2, '2023-10-15 15:30:00', 1500.00), (3, '2023-10-20 18:00:00',
500.00), (6, '2023-10-25 12:00:00', 4500.00),

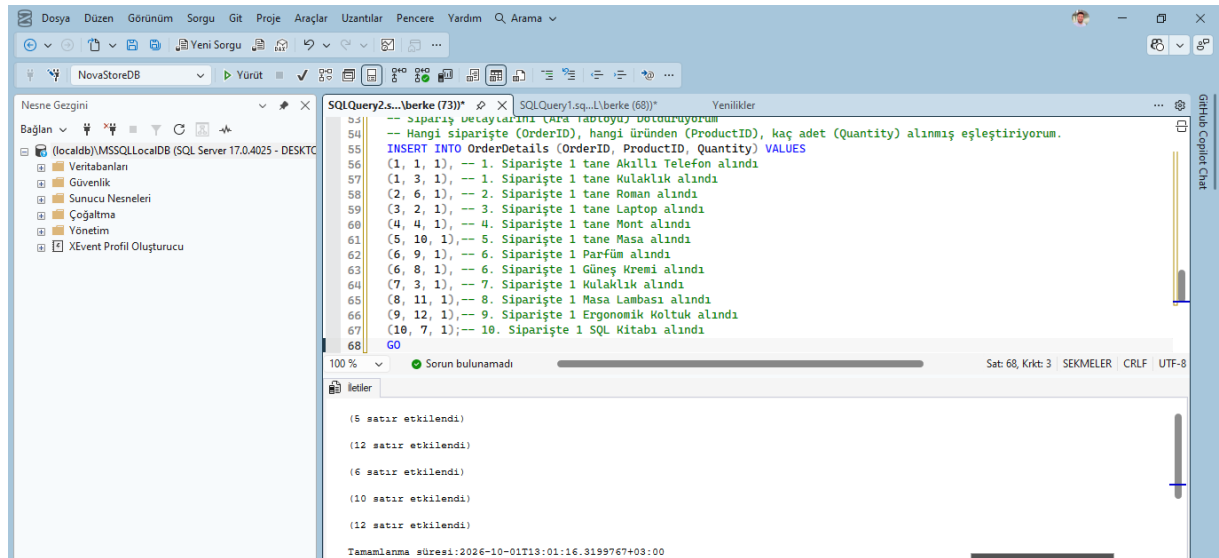
(1, '2023-10-28 17:30:00', 250.00);

-- Son olarak hangi siparişte, hangi üründen kaç tane alındığını
eşleştiriyorum (Sipariş Detayları).

INSERT INTO OrderDetails (OrderID, ProductID, Quantity) VALUES

(1, 1, 1), (1, 3, 1), (2, 6, 1), (3, 2, 1), (4, 4, 1), (5, 10, 1),
(6, 9, 1), (6, 8, 1), (7, 3, 1), (8, 11, 1), (9, 12, 1), (10, 7, 1);

```



BÖLÜM 3: Sorgulama ve Analiz (DQL - Select ve Joins)

1. Temel Listeleme

Soru: Stok miktarı 20'den az olan ürünlerin adını ve stok miktarını "AZALAN" sırada listele.

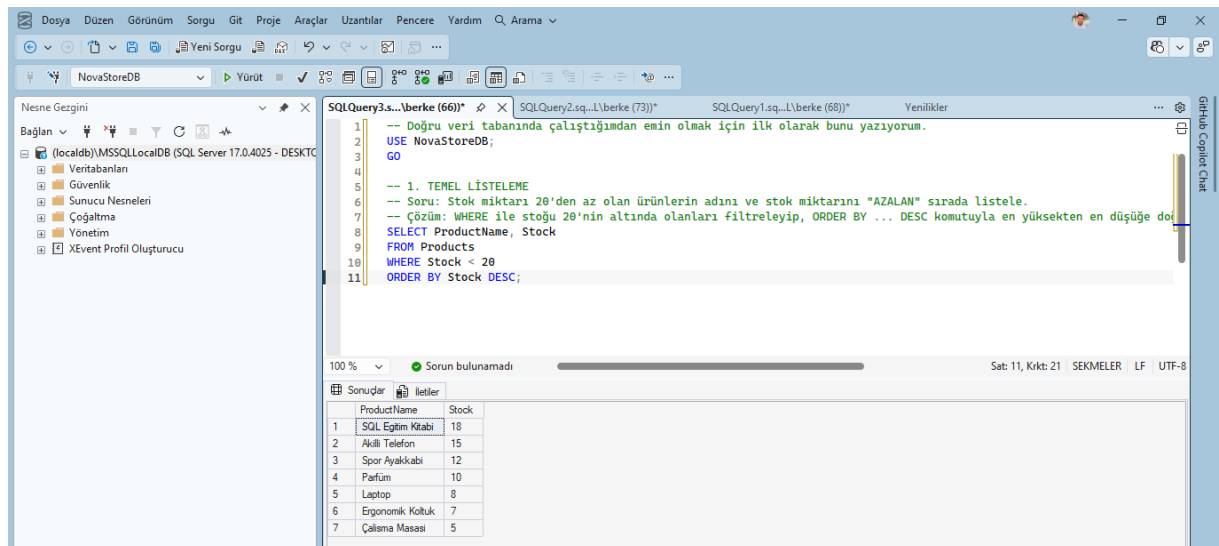
-- Stoğu 20'nin altında kalan kritik ürünleri WHERE ile filtreleyip, en yüksekten en düşüğe (DESC) diziyorum.

SELECT ProductName, Stock

FROM Products

WHERE Stock < 20

ORDER BY Stock DESC;



	ProductName	Stock
1	SQL Egitim Kitabı	18
2	Akıllı Telefon	15
3	Spor Ayakkabı	12
4	Parfüm	10
5	Laptop	8
6	Ergonomik Koltuk	7
7	Çalışma Masası	5

Açıklama: Kritik seviyeye düşen ürünleri tespit etmek için WHERE ile 20'den az stokları filtreledim ve ORDER BY DESC kullanarak en yüksek stoktan en düşüğe doğru sıraladım.

2. Veri Birleştirme (JOIN)

Soru: Hangi müşteri hangi tarihte sipariş vermiş?

-- Hangi müşterinin ne zaman sipariş verdiğini görmek için Customers ve Orders tablolarını ID'lerinden birleştiriyorum.

```
SELECT c.FullName, c.City, o.OrderDate, o.TotalAmount
FROM Customers c
INNER JOIN Orders o ON c.CustomerID = o.CustomerID;
```

The screenshot shows the SQL Server Enterprise Manager interface. The query window displays the following SQL code:

```
7 -- Çözüm: WHERE ile stoğu 20'nin altında olanları filtreleyip, ORDER BY ... DESC komutuyla en yüksekten en düşüğe do
8 SELECT ProductName, Stock
9 FROM Products
10 WHERE Stock < 20
11 ORDER BY Stock DESC;
12
13 -- 2. VERİ BİRLEŞTİRME (JOIN)
14 -- Soru: Hangi müşteri hangi tarihte sipariş vermiş? (Müşteri Adı, Şehir, Sipariş Tarihi, Toplam Tutar)
15 -- Çözüm: Müşteriler ve Siparişler tablolarını INNER JOIN ile CustomerID üzerinden birleştiriyorum.
16 SELECT c.FullName, c.City, o.OrderDate, o.TotalAmount
17 FROM Customers c
18 INNER JOIN Orders o ON c.CustomerID = o.CustomerID;
```

The results grid shows the following data:

FullName	City	OrderDate	TotalAmount
3 Ayşe Kaya	Ankara	2023-10-02 09:15:00.000	45000.00
4 Mehmet De...	İzmir	2023-10-03 16:45:00.000	2500.00
5 Fatma Çelik	Bursa	2023-10-10 11:00:00.000	3000.00
6 Ali Can	Antalya	2023-10-12 13:20:00.000	1600.00
7 Ayşe Kaya	Ankara	2023-10-15 15:30:00.000	1500.00
8 Mehmet De...	İzmir	2023-10-20 18:00:00.000	500.00
9 Zeynep Sa...	Eski...	2023-10-25 12:00:00.000	4500.00
10 Ahmet Yılmaz	İstanbul	2023-10-28 17:30:00.000	250.00

	FullName	City	OrderDate	TotalAmount
1	Ahmet Yılmaz	İstanbul	2023-10-01 10:30:00.000	26500.00
2	Ahmet Yılmaz	İstanbul	2023-10-05 14:00:00.000	150.00
3	Ayşe Kaya	Ankara	2023-10-02 09:15:00.000	45000.00
4	Mehmet De...	İzmir	2023-10-03 16:45:00.000	2500.00
5	Fatma Çelik	Bursa	2023-10-10 11:00:00.000	3000.00
6	Ali Can	Antalya	2023-10-12 13:20:00.000	1600.00
7	Ayşe Kaya	Ankara	2023-10-15 15:30:00.000	1500.00
8	Mehmet De...	İzmir	2023-10-20 18:00:00.000	500.00

Açıklama: Müşterilerin sipariş geçmişini görebilmek için Customers ve Orders tablolarını CustomerID ortak alanı üzerinden INNER JOIN ile birleştirdim.

3. Çoklu Birleştirme ve Detay Raporu

Soru: "Ahmet Yılmaz" isimli müşterinin aldığı ürünlerin isimlerini, fiyatlarını ve kategorilerini listele.

-- Ahmet Yılmaz'ın aldığı ürünlerin en ince detayına inmek için 5 tabloyu aralarındaki ortak anahtarlarla zincirleme birbirine bağlıyorum.

```
SELECT c.FullName, p.ProductName, p.Price, cat.CategoryName
FROM Customers c
JOIN Orders o ON c.CustomerID = o.CustomerID
JOIN OrderDetails od ON o.OrderID = od.OrderID
JOIN Products p ON od.ProductID = p.ProductID
JOIN Categories cat ON p.CategoryID = cat.CategoryID
WHERE c.FullName = 'Ahmet Yılmaz';
```

The screenshot shows the SQL Server Enterprise Manager interface. The query editor displays the following SQL query:

```
-- Soru: hangi müşteri hangi tarihte sipariş vermiş? (müşteri adı, şehir, sipariş tarihini, toplam tutarı)
-- Çözüm: Müşteriler ve Siparişler tablolarını INNER JOIN ile CustomerID üzerinden birleştiriyorum.
SELECT c.FullName, c.City, o.OrderDate, o.TotalAmount
FROM Customers c
INNER JOIN Orders o ON c.CustomerID = o.CustomerID;

-- 3. ÇOKLU BİRLEŞTİRME VE DETAY RAPORU
-- Soru: "Ahmet Yılmaz" isimli müşterinin aldığı ürünlerin isimlerini, fiyatlarını ve kategorilerini listele.
-- Çözüm: Müşteriden başlayıp ürün kategorisine kadar tam 5 tabloyu aralarındaki Foreign Key'ler sayesinde zincirlem
SELECT c.FullName, p.ProductName, p.Price, cat.CategoryName
FROM Customers c
JOIN Orders o ON c.CustomerID = o.CustomerID
JOIN OrderDetails od ON o.OrderID = od.OrderID
JOIN Products p ON od.ProductID = p.ProductID
JOIN Categories cat ON p.CategoryID = cat.CategoryID
WHERE c.FullName = 'Ahmet Yılmaz';
```

The results pane shows the following data:

FullName	ProductName	Price	CategoryName
Ahmet Yılmaz	Akıllı Telefon	25000.00	Elektronik
Ahmet Yılmaz	Bluetooth Kulaklık	1500.00	Elektronik
Ahmet Yılmaz	Bilin Kurgu Rom...	150.00	Kıtap
Ahmet Yılmaz	SQL Eğitim Kitabı	250.00	Kıtap

	FullName	ProductName	Price	CategoryName
1	Ahmet Yılmaz	Akıllı Telefon	25000.00	Elektronik
2	Ahmet Yılmaz	Bluetooth Kulaklık	1500.00	Elektronik
3	Ahmet Yılmaz	Bilim Kurgu Rom...	150.00	Kitap
4	Ahmet Yılmaz	SQL Eğitim Kitabı	250.00	Kitap

Açıklama: Ahmet Yılmaz'ın aldığı ürünlerin en ince detayına kadar inmek için Müşteriler tablosundan başlayıp Kategoriler tablosuna kadar tam 5 tabloyu aralarındaki Foreign Key'ler sayesinde zincirleme birbirine bağladım.

4. Gruplama ve Aggregate Fonksiyonlar

Soru: Hangi kategoride toplam kaç adet ürünümüz var?

-- Kategorileri gruplayıp içindeki ürünleri saydırıyorum. Hiç ürünü olmayan kategoriler de listede görünsün diye LEFT JOIN kullandım.

```
SELECT cat.CategoryName, COUNT(p.ProductID) AS ToplamUrunSayisi
FROM Categories cat
LEFT JOIN Products p ON cat.CategoryID = p.CategoryID
GROUP BY cat.CategoryName;
```

The screenshot shows the SQL Server Enterprise Manager interface. The query editor displays the following SQL code:

```
-- 4. GRUPLAMA VE AGGREGATE FONKSİYONLAR
-- Soru: Hangi kategoride toplam kaç adet ürünümüz var?
-- Çözüm: Kategorileri GROUP BY ile gruplayıp, her birinin içine düşen ürünleri COUNT fonksiyonuyla saydırıyorum. (H:
SELECT cat.CategoryName, COUNT(p.ProductID) AS ToplamUrunSayisi
FROM Categories cat
LEFT JOIN Products p ON cat.CategoryID = p.CategoryID
GROUP BY cat.CategoryName;
```

The results pane shows the following data:

FullName	ProductName	Price	CategoryName	
1	Ahmet Yılmaz	Akıllı Telefon	25000.00	Elektronik
2	Ahmet Yılmaz	Bluetooth Kulaklık	1500.00	Elektronik
3	Ahmet Yılmaz	Bilim Kurgu Rom...	150.00	Kitap
4	Ahmet Yılmaz	SQL Eğitim Kitabı	250.00	Kitap

Below the main results, there is a summary table:

CategoryName	ToplamUrunSayisi	
1	Elektronik	3
2	Ev ve Yaşam	3
3	Giyim	2
4	Kitap	2
5	Kozmetik	2

	CategoryName	ToplamUrunSayisi
1	Elektronik	3
2	Ev ve Yaşam	3
3	Giyim	2
4	Kitap	2
5	Kozmetik	2

Açıklama: Kategorilerimi GROUP BY ile gruplandırımdım ve her birinin içine düşen ürünleri COUNT ile saydım. Hiç ürünü olmayan kategoriler de listede görünsün diye LEFT JOIN kullanmayı tercih ettim.

5. Ciro Analizi (Zor)

Soru: Her müşterinin şirkete kazandırdığı toplam ciro nedir? En çok harcama yapandan en aza doğru sırala.

-- Kime ne kadar satış yaptığımızı bulmak için sipariş tutarlarını SUM ile toplayıp, en çok harcadıyandan en aza doğru sıralıyorum.

```
SELECT c.FullName, SUM(o.TotalAmount) AS ToplamCiro
FROM Customers c
JOIN Orders o ON c.CustomerID = o.CustomerID
GROUP BY c.FullName
ORDER BY ToplamCiro DESC;
```

FullName	ToplamCiro
1. Ayşe Kaya	46500.00
2. Ahmet Yılmaz	26900.00
3. Zeynep Sa...	4500.00
4. Fatma Çelik	3000.00
5. Mehmet De...	3000.00
6. Ali Can	1600.00

	FullName	ToplamCiro
1	Ayşe Kaya	46500.00
2	Ahmet Yılmaz	26900.00
3	Zeynep Sa...	4500.00
4	Fatma Çelik	3000.00
5	Mehmet De...	3000.00
6	Ali Can	1600.00

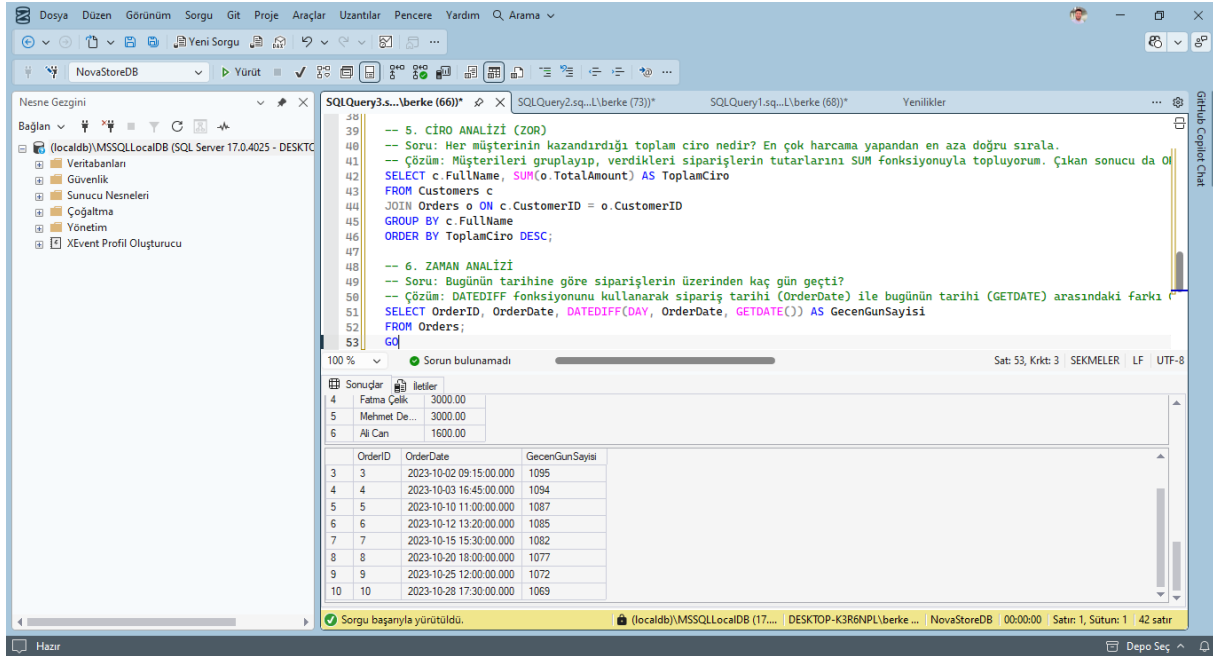
Açıklama: Şirketime en çok para kazandıran müşterileri bulmak için sipariş tutarlarını SUM fonksiyonuyla toplayıp, sonuçları ORDER BY DESC ile büyükten küçüğe sıraladım.

6. Zaman Analizi

Soru: Bugünün tarihine göre siparişlerin üzerinden kaç gün geçti?

-- Siparişlerin üzerinden kaç gün geçtiğini hesaplamak için DATEDIFF fonksiyonu ile sipariş tarihinden bugünün tarihini çıkartıyorum.

```
SELECT OrderID, OrderDate, DATEDIFF(DAY, OrderDate, GETDATE()) AS GecenGunSayisi  
FROM Orders;
```



	OrderID	OrderDate	GecenGunSayisi
1	1	2023-10-01 10:30:00.000	1096
2	2	2023-10-05 14:00:00.000	1092
3	3	2023-10-02 09:15:00.000	1095
4	4	2023-10-03 16:45:00.000	1094
5	5	2023-10-10 11:00:00.000	1087
6	6	2023-10-12 13:20:00.000	1085
7	7	2023-10-15 15:30:00.000	1082
8	8	2023-10-20 18:00:00.000	1077

Açıklama: DATEDIFF fonksiyonunu kullanarak siparişlerin verildiği tarih ile bugünün tarihi (GETDATE()) arasındaki farkı gün (DAY) cinsinden hesaplattım.

BÖLÜM 4: İleri Seviye Veri Tabanı Nesneleri

-- View (Görünüm) Oluşturma

-- Sürekli upuzun JOIN sorguları yazıp kafamı karıştırmamak için sık kullandığım bilgileri tek bir sanal tabloda (View) topluyorum.

```
CREATE VIEW vw_SiparisOzet AS
```

```
SELECT c.FullName, o.OrderDate, p.ProductName, od.Quantity
```

```
FROM Customers c
```

```
JOIN Orders o ON c.CustomerID = o.CustomerID
```

```
JOIN OrderDetails od ON o.OrderID = od.OrderID
```

```
JOIN Products p ON od.ProductID = p.ProductID;
```

```
GO
```

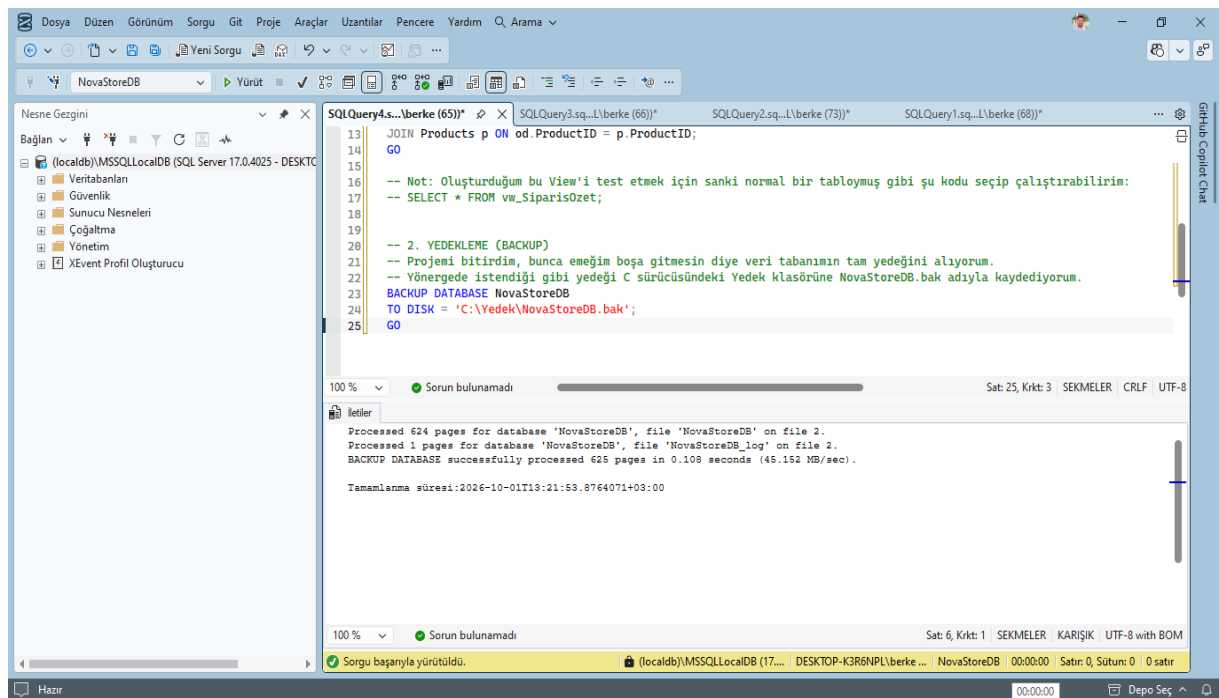
-- Yedekleme (Backup)

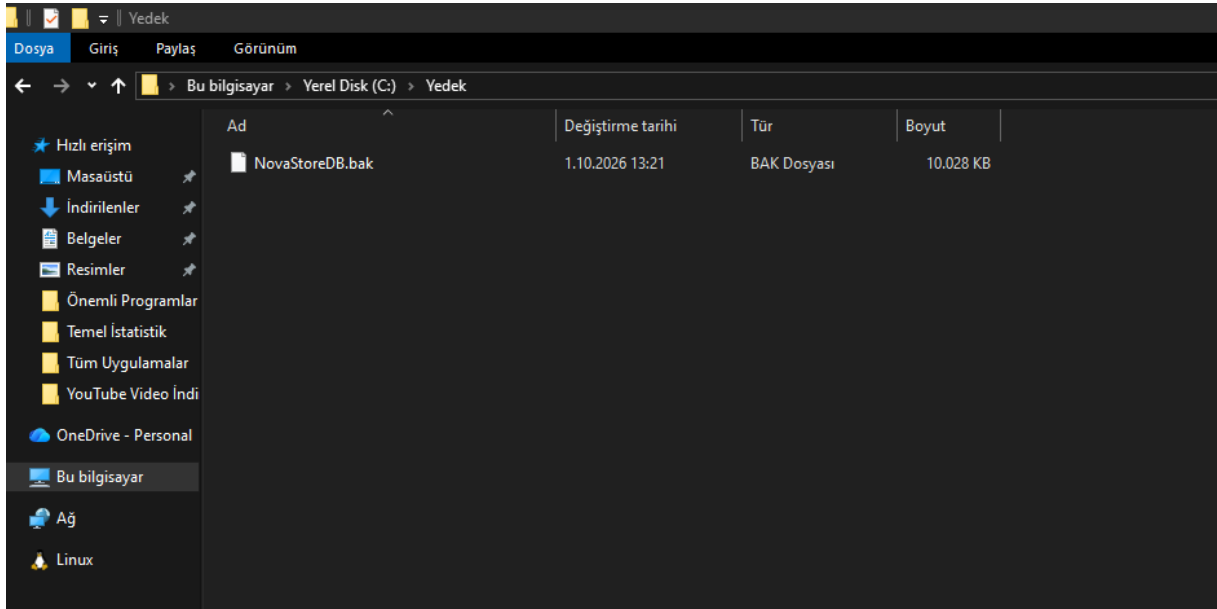
-- Onca emeğim boşa gitmesin diye veri tabanımın tam yedeğini bilgisayarımdaki C:\Yedek klasörüne alıyorum.

```
BACKUP DATABASE NovaStoreDB
```

```
TO DISK = 'C:\Yedek\NovaStoreDB.bak';
```

```
GO
```





Açıklama: Sürekli upuzun JOIN sorguları yazmamak için sık kullandığım müşteri, sipariş ve ürün verilerini vw_SiparisOzet isimli tek bir sanal tabloda (View) topladım. Son olarak da tüm emeğimin kaybolmaması için veri tabanımın yedeğini C:\Yedek klasörüne NovaStoreDB.bak adıyla çıkardım.

BÖLÜM 5: Veri Tabanı Diyagramı (İlişkisel Şema)

Tablolarımın Primary Key ve Foreign Key mantığıyla birbirine nasıl bağlandığını gösteren ilişkisel şema (Database Diagram) aşağıdadır:

